

MANITRA

Analyse du fichier config/database.php

✓ Tes points forts

Encapsulation correcte : L'utilisation de propriétés privées (private) pour les identifiants est une excellente pratique de sécurité.

Configuration du Charset : Tu as bien inclus charset=utf8 directement dans ton DSN. C'est parfait pour garantir que les noms de tes contacts s'affichent correctement sans erreurs de caractères spéciaux.

Gestion des erreurs : L'activation de PDO::ATTR_ERRMODE en mode ERRMODE_EXCEPTION est très bien. Cela permet d'intercepter les erreurs SQL proprement au lieu de laisser le script continuer avec des données invalides.

Tes points à améliorer

Instance de connexion : Ta méthode connect() renvoie un nouvel objet PDO à chaque appel. Comme pour certains de tes camarades, sur un projet plus vaste, il vaudrait mieux stocker la connexion dans une propriété pour la réutiliser durant toute l'exécution de la page afin d'économiser les ressources du serveur.

Sécurité du die() : Afficher directement \$e->getMessage() est pratique pour le développement, mais risqué en production car cela peut révéler des informations sur ton infrastructure (nom d'utilisateur, structure de la base) en cas de panne.

Analyse de ton fichier `models/Contact.php`

✓ Tes points forts

Gestion du constructeur : C'est impeccable. En instanciant `new Database()` et en récupérant la connexion dès le départ, tu simplifies toutes les autres méthodes.

Utilisation de `fetchAll(PDO::FETCH_ASSOC)` : Contrairement à `Hasina` ou `Mahefa`, tu renvoies directement un tableau associatif. C'est la meilleure approche pour le MVC car ton contrôleur n'a pas besoin de savoir ce qu'est un objet PDO pour manipuler les données.

Sécurité SQL : Tu utilises les requêtes préparées avec des marqueurs nommés (`:name`, `:email`) dans tes méthodes `create` et `delete`. C'est la protection standard contre les injections SQL.

Syntaxe moderne : Le passage du tableau associatif directement dans la méthode `execute()` est une syntaxe PHP très propre qui évite d'écrire plusieurs lignes de `bindParam`.

Tes points à améliorer

Validation des types : Tu ne précises pas les types des arguments (ex: `string $name`, `int $id`). Ajouter ces types permettrait à PHP de bloquer les erreurs avant même qu'elles n'arrivent à la base de données.

Sécurité de la requête `getAll()` : Pour cette méthode, tu utilises `$this->conn->query()`. Bien que ce ne soit pas dangereux ici car il n'y a pas de variables utilisateur, il est souvent recommandé de rester sur du `prepare()` par pure habitude de sécurité.

Analyse de ton ContactController.php

✓ Tes points forts

Logique de délégation : C'est exactement ce qu'on attend en MVC. Ton contrôleur ne touche pas à la base de données, il donne simplement des ordres à `$this->model`.

Redirections propres : L'utilisation de `header("Location: ...")` après les actions `add` et `delete` est la meilleure pratique pour éviter que l'utilisateur ne soumette deux fois le même formulaire par erreur en rafraîchissant sa page.

Gestion des méthodes HTTP : Ta méthode `add` vérifie bien si la requête est de type `POST` avant de tenter de traiter les données.

Tes points à améliorer

Validation des données : Bien que tu vérifies si les champs ne sont pas vides (`!empty`), il manque une validation du format de l'adresse email (avec `filter_var`).

Sécurité de l'ID : Pour la méthode `delete`, tu passes `$_GET['id']` directement au modèle. Il serait plus prudent de forcer la conversion en entier (`((int)$_GET['id'])`) pour s'assurer qu'aucune donnée malveillante n'est transmise.

Gestion du "View Rendering" : Ta méthode `list` inclut la vue directement. C'est correct, mais attention à la portée des variables. Ici, `$contacts` est bien accessible par `views/contacts.php`, ce qui est parfait.

Analyse de ton fichier `views/contacts.php`

✓ Tes points forts

Sécurité (Protection XSS) : C'est parfait. Tu as systématiquement utilisé `htmlspecialchars()` pour l'affichage des données provenant de la base. C'est la règle d'or pour éviter les failles de sécurité, et tu l'as respectée à la lettre.

Expérience Utilisateur (UX) : Tu as intégré une confirmation de suppression en JavaScript (`onclick="return confirm(...)"`). C'est un petit détail qui évite bien des frustrations aux utilisateurs distraits.

Structure HTML/CSS : Ton code est propre, bien indenté et le CSS interne permet d'avoir un tableau lisible sans surcharger le projet de fichiers externes. L'utilisation de `border-collapse` rend le tableau bien plus esthétique.

Logique de boucle : La syntaxe alternative de PHP (`foreach : ... endforeach;`) est idéale dans une vue car elle reste très lisible au milieu des balises HTML.

Tes points à améliorer

Accessibilité : Tes champs de saisie (`input`) n'ont pas de balises `<label>`. Pour les lecteurs d'écran ou simplement pour le confort de clic, il est toujours préférable de lier un label à chaque champ.

Gestion des retours : Ton formulaire pointe vers `index.php?action=add`. C'est correct, mais comme ton contrôleur redirige immédiatement après l'ajout, l'utilisateur ne voit pas de message de succès. Ajouter un petit bandeau "Contact ajouté avec succès" serait un vrai plus.

Fiche d'Évaluation : Projet Répertoire MVC

Étudiant : Manitra

Projet : Gestionnaire de Contacts (Style Minimaliste)

Note Globale : 17,5 / 20

Détail de l'Évaluation

Critère	Note	Observations
Architecture MVC	5 / 5	Découpage impeccable. Le modèle est autonome et le contrôleur est parfaitement léger.
Sécurité & Rigueur	4,5 / 5	Excellent usage de PDO et protection XSS systématique. Seule la validation du format d'email manque à l'appel.
Qualité du Code (PHP)	4,5 / 5	Code moderne, clair et très facile à relire. L'usage des tableaux associatifs simplifie tout.
Interface & UX (UI)	3,5 / 5	Interface sobre et efficace, mais manque un peu de "caractère" ou de retour visuel après action.